

```

"""
CoachIA Foot – Pipeline de génération de séance
=====

Entrée  : profil du club + contexte de la semaine (effectif, adversaire, objectif
Sortie  : PDF de séance brandé, prêt à imprimer

Étapes :
1. Appel API Claude -> génère le contenu de la séance en JSON strict
2. Injection du JSON dans le template HTML paramétrique
3. Rendu PDF via wkhtmltopdf
4. Vérification visuelle optionnelle via pdf2image
"""

import json
import subprocess
import datetime
from pathlib import Path
import anthropic

client = anthropic.Anthropic() # clé API lue depuis l'environnement

TEMPLATE_PATH = Path("template_seance.html")
OUTPUT_DIR = Path("output")
OUTPUT_DIR.mkdir(exist_ok=True)

# -----
# 1. Génération du contenu de la séance via l'IA
# -----

def generer_contenu_seance(club: dict, contexte_semaine: dict, historique: list)
    """
    Appelle Claude pour générer une séance adaptée.
    Renvoie un JSON strict (pas de texte libre) pour pouvoir l'injecter
    directement dans le template sans parsing fragile.
    """

    system_prompt = """Tu es un entraîneur de football expérimenté qui conçoit de
d'entraînement. Tu dois répondre UNIQUEMENT avec un objet JSON valide, sans
texte avant ou après, sans balises markdown. Le format attendu est strictement :

{
  "objectif_semaine": "phrase courte décrivant l'objectif",
  "exercices": [

```

```

    {
        "duree_min": 15,
        "titre": "Nom de l'exercice",
        "categorie": "Activation | Passes | Finition | Possession | Tactique",
        "consigne": "description concrète et actionnable, 2-3 phrases max"
    }
]
}

```

Règles :

- La somme des durées doit correspondre à la durée totale de séance demandée.
- Ne jamais proposer un exercice déjà présent dans l'historique récent fourni.
- Adapter la difficulté et l'intensité à l'effectif disponible indiqué.
- Si un adversaire est précisé, orienter au moins un exercice vers cette préparation (bloc défensif, transitions, etc.)."""

```

user_prompt = f"""Club : {club['nom']], formation habituelle {club['formation']}
Effectif disponible cette semaine : {contexte_semaine['effectif_dispo']} joueurs
Durée totale de la séance : {contexte_semaine['duree_totale_min']} minutes
Prochain adversaire : {contexte_semaine.get('adversaire', 'non précisé')}
Exercices déjà travaillés récemment (à éviter) : {historique}

```

Génère la séance de la semaine."""

```

response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=2000,
    system=system_prompt,
    messages=[{"role": "user", "content": user_prompt}],
)

raw_text = response.content[0].text.strip()
# sécurité : au cas où le modèle ajoute des balises malgré la consigne
raw_text = raw_text.replace("`json", "").replace("`", "").strip()

return json.loads(raw_text)

# -----
# 2. Construction du bloc HTML des exercices
# -----

def construire_blocs_exercices(exercices: list) -> str:
    blocs = []
    for ex in exercices:
        bloc = f"""
        <div class="exercice">

```

```

        <div class="duree">{ex['duree_min']}'</div>
        <div class="detail">
            <span class="categorie">{ex['categorie']}'</span>
            <span class="titre">{ex['titre']}'</span>
            <span class="consigne">{ex['consigne']}'</span>
        </div>
    </div>""
    blocs.append(bloc)
return "\n".join(blocs)

# -----
# 3. Injection dans le template + génération du PDF
# -----

def generer_pdf(club: dict, contenu_seance: dict, contexte_semaine: dict) -> Path
    html = TEMPLATE_PATH.read_text(encoding="utf-8")

    remplacements = {
        "{{club_nom}}": club["nom"] or "Mon Club",
        "{{logo_url}}": club.get("logo_url") or "https://placeholder.co/80x80?text=",
        "{{couleur_primaire}}": club["couleur_primaire"] or "#0B2545",
        "{{couleur_secondaire}}": club["couleur_secondaire"] or "#2E8B57",
        "{{formation}}": club["formation"] or "4-3-3",
        "{{effectif_dispo}}": str(contexte_semaine["effectif_dispo"]),
        "{{date_seance}}": contexte_semaine["date_seance"],
        "{{objectif_semaine}}": contenu_seance["objectif_semaine"],
        "{{blocs_exercices}}": construire_blocs_exercices(contenu_seance["exercic
        "{{date_generation}}": datetime.date.today().strftime("%d/%m/%Y"),
    }

    for cle, valeur in remplacements.items():
        html = html.replace(cle, str(valeur))

    slug = club["nom"].lower().replace(" ", "_")
    html_path = OUTPUT_DIR / f"seance_{slug}.html"
    pdf_path = OUTPUT_DIR / f"seance_{slug}.pdf"
    html_path.write_text(html, encoding="utf-8")

    # Commande confirmée dans ton workflow existant : marges zéro, format A4
    subprocess.run([
        "wkhtmltopdf",
        "--page-size", "A4",
        "--margin-top", "0", "--margin-bottom", "0",
        "--margin-left", "0", "--margin-right", "0",
        "--enable-local-file-access",
        str(html_path), str(pdf_path)
    ])

```

```

], check=True)

return pdf_path

# -----
# 4. Vérification visuelle (optionnelle, reprend ton usage de pdf2image)
# -----

def verifier_visuellement(pdf_path: Path):
    from pdf2image import convert_from_path
    images = convert_from_path(str(pdf_path), dpi=100)
    preview_path = pdf_path.with_suffix(".png")
    images[0].save(preview_path)
    return preview_path

# -----
# Exemple d'utilisation (ce que le formulaire web appellerait)
# -----

if __name__ == "__main__":
    club_exemple = {
        "nom": "Diogountouro FC",
        "logo_url": "https://exemple.com/logo_diogou.png",
        "couleur_primaire": "#0B2545",
        "couleur_secondaire": "#2E8B57",
        "formation": "4-3-3",
    }

    contexte_semaine = {
        "effectif_dispo": 16,
        "duree_totale_min": 90,
        "date_seance": "02/08/2026",
        "adversaire": "FC Rivalis",
    }

    historique_recent = ["Rondo 4v2", "Possession plein terrain"]

    contenu = generer_contenu_seance(club_exemple, contexte_semaine, historique_r
    pdf = generer_pdf(club_exemple, contenu, contexte_semaine)
    print(f"PDF généré : {pdf}")

```